

PLPP Assignment 1

Matvii Kovalyshyn

<https://github.com/VY-Assignments/text-editor-matvii-kovalyshyn>

Program explanation

- Top-down decomposition: how you broke the editor into procedures/functions, and where you stopped subdividing

Code is divided into a number of functions, each for completing one task. They include functions for creating array, working with it, file saving and loading. In this case, function main has only user interface and functions calls. I stop subdividing when the code is already concise and well readable, and functions focus on one task.

- Prove structured & procedural paradigms usage

Structured and procedural paradigms are crucial in this project. Structured includes if- and for-constructions, which are the core of the code. Procedural paradigm is shown in segmentation code into blocks, which are called functions or procedures. Without using this paradigm code will be unstructured and hard to read and work with.

- Data structure choice: why dynamic array or linked list for lines/chars

For data structure, dynamic array was chosen because of its faster access to elements by index and saving data in one place in the memory. In functions dynamic array has the same or sometimes better memory complexity in most cases compared to linked list.

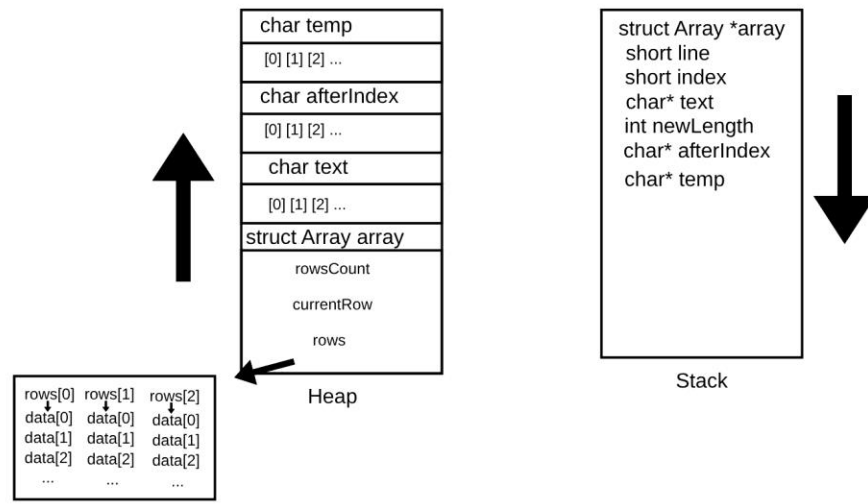
- Memory reallocation logic for insert

*First, we find the length of a future row by summing the length of given row and inserted text. This length we use when selecting memory: realloc(startRow, newLength * sizeof(char)) for temporary array in case there are problems with memory. If there are no errors, we return to our row.*

Memory layout diagram

- Show where your buffer lives (heap via malloc/realloc in RAM)
- Show a stack frame during a function call (what's on the runtime stack)

Taking Insert function as an example.



Test plan — one screenshot per scenario

- Append text → print (happy path)

```
Choose the command: 1
Enter text to append: hello world

Supported commands.
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.

Choose the command: 5

Text:
hello world
```

- New line → print (multiline display)

```
Choose the command: 2
New line is started.
Supported commands.
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.
```

```
Choose the command: 5
```

```
Text:
hello world
```

- Save → load → print (file round-trip)

```
Choose the command: 3
Enter the file name for saving: MyText.txt
hello world
```

```
Text has been saved successfully.
```

```
Supported commands.
```

```
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.
```

```
Choose the command: 4
```

```
Enter the file name for loading: MyText.txt
```

```
Text has been loaded successfully.
```

```
Supported commands.
```

```
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.
```

```
Choose the command: 5
```

```
Text:
hello world
```

- Insert → print (matches assignment example)

```
Choose the command: 6
Choose line and index: 0 6
Enter text to insert: my
```

Supported commands.

1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.

```
Choose the command: 5
```

```
Text:
hello my world
```

- Search with one match, search with no match ("not found")

```
Choose the command: 7
Enter text to search: my
Text is present in this position: (0, 6)
Supported commands.
1. Append to end.
2. New line.
3. Save to file.
4. Load from file.
5. Print to console.
6. Insert.
7. Search.
8. Exit.

Choose the command: 7
Enter text to search: myText
Text is not found.
```

- Edge cases (from quality acceptance criteria):

- Empty buffer: print, save, search without crash

- Print:

```
Choose the command: 5

Text:
```

- Save:

```
Choose the command: 3
Enter the file name for saving: MyText.txt
Text has been saved successfully.
```

```
Choose the command: 3
Enter the file name for saving:
Incorrect file name.
```

- Search:

```
Choose the command: 7
Enter text to search:
Text is not found.
```

- Load non-existent file: error message, buffer unchanged

```
Choose the command: 4
Enter the file name for loading: 123
Error opening file.
```

```
Choose the command: 5

Text:
hello world
```

- Insert with out-of-range line/column: explicit error

```
Choose the command: 6
Choose line and index: 100 100
Enter text to insert: text
Invalid line or index.
```

- Append past initial allocation: error or buffer growth (extra point)

Buffer grows if we want to add over the current capacity of the array. New length will be double increased through realloc if we run out of lines. If we reach capacity in a row, we calculate new length by adding current length, length of a new text and 1 for '\0'.

Answers to control questions

1. What problem did structured programming solve? Why was the unrestricted goto considered harmful?

Structured programming added construction for, while, if and similar. This makes code more readable and run in a sequence. With goto it was hard to track and debug program status.

2. What is the structured programming paradigm?

Structured paradigm is a kind of imperative paradigm, which added constructions according to 3 principles: sequence, selection and repetition.

3. Name the three control-flow constructs that structured programming claims are sufficient.

Give a short example of each in your editor.

For loop (used in Search for comparing chars)

While loop (used for user interface in main)

If (used in almost each function, for example, error handling)

4. What is the procedural programming paradigm?

Procedural paradigm is a kind of imperative paradigm, which focuses on procedures (functions).

5. Explain the parameter-passing modes (by value, by reference, by result, by name). Which of these does C support natively, and how do you simulate the others?

Parameter-passing modes are the way function works with arguments.

By value: creates a new individual copy, uses for ordinary data types.

By reference: gets a reference using pointers to the same memory location, uses for array, struct and other complex data types.

By result: creates an empty copy, which will overwrite original variable after function block.

By name: the argument is called every time it is used in function and the value depends on when it was called.

In C the most common parameter-passing modes are by value and by reference.

6. Top-down decomposition: walk through how you broke the editor down into procedures.

Where did you stop subdividing, and why?

Code is divided into a number of functions, each for completing one task. They include functions for creating array, working with it, file saving and loading. In this case, function main has only user interface and functions calls. I stop subdividing when the code is already concise and well readable, and functions focus on one task.

7. Which primitive data types do you know? What are the key differences between them?

Int – for whole numbers.

Float, double – for fractional numbers.

Bool – for 1 or 0 (true or false).

Char – for any character.

And there are a lot of variations of these types.

The main difference is the memory capacity of each type and what it is used for.

8. Which segments does process memory have?

Stack is used for local variables and function arguments.

Heap is used for complex data types by using malloc.

Data is used for static data.

Code is used for saving your code to run.

9. What is the virtual memory space?

Virtual memory space is memory, which is connected through virtual addresses to the physical addresses before it can be accessed and used.

10. How do you allocate the memory in each segment?

In stack you allocate memory by creating simple variables, sizes of which are static and known. In heap you use malloc(size) and choose how much memory you need.

11. How do you reallocate the memory in case we don't have enough space?

For reallocating memory we can use realloc(previousVariable, newSize). First, you need to know how much memory you need to increase.

12. What is the runtime stack?

Runtime stack is a process memory segment, that works on the principle LIFO (Last In – First Out) and is used to store local declared variables and function arguments.

Git history screenshot (git log / github / gitk)

